

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	T Bhanu Teja	Reg. No.:	192472259
Section / Batch:	2024	Date:	25-08-26

Code of Conduct

I T Bhanu Teja(192472259) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bhanu Teja

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

Context-Free Grammar (CFG) / Constituency Parsing

CFG focuses on how words group together to form larger units called constituents. It is based on formal grammar rules such as S -> NP VP. This approach is highly structural and recursive, making it ideal for grammar checking software. For example, in the sentence 'The cat sat', CFG identifies 'The cat' as a Noun Phrase and 'sat' as a Verb Phrase. It excels at identifying the nesting of language but often fails to explicitly show the direct relationship between a verb and its objects across long distances.

2. Dependency Parsing

Dependency parsing focuses on the functional relationships between words (head-dependent). There are no phrasal nodes. Instead, every word is linked to a 'head' that it modifies. For example, 'cat' depends on 'sat' as the subject. This is much more effective for 'free word order' languages and for downstream semantic tasks like information extraction, as it directly connects the action to the actor.

3. Comparative Analysis & Flow Chart

Flow Chart: [Sentence] -> [Syntactic Analysis] -> (Choice A: CFG -> Grouping by Phrases) / (Choice B: Dependency -> Linking Word to Word).

Feature	CFG Trees	Dependency Structure
Focus	Hierarchical grouping (Constituents)	Functional relations (Head-Child)
Complexity	High (many non-terminal nodes)	Lower (nodes = words)
Application	Formal grammar verification	Semantic role labeling / IE

4. Justification

Dependency parsing is more effective for capturing meaning because it abstracts away from the specific phrase structure and highlights the core semantic ties. In a system requiring syntactic analysis for meaning, dependency paths provide a shorter and more direct route to understanding 'who did what to whom' than a complex constituent tree.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.

Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

For real-time applications like voice assistants or chatbots, the speed and robustness of the parser are paramount, especially when dealing with user input that might be messy or incomplete.

1. Top-Down Parsing (Recursive Descent)

Top-down parsing starts with the highest-level rule (S) and tries to predict what words should appear. It is intuitive but has significant flaws: it cannot handle left-recursive grammars (it enters an infinite loop) and it involves massive amounts of backtracking. If a sentence is ambiguous, it may try thousands of combinations before finding the right one, making it too slow for real-time dynamic inputs.

2. Earley Parsing (Dynamic Programming)

Earley parsing uses a 'chart' to store partial results. It processes a sentence in a single pass ($O(n^3)$ complexity). It uses three operations: PREDICTOR, SCANNER, and COMPLETER. By storing the state of the parse at each word, it avoids repeating work. Even if a sentence is incomplete, the Earley chart contains all possible valid partial structures up to that point.

3. Comparative Flow Chart

Flow Chart: [User Input] -> [Parser] -> (If Top-Down: Predict S -> Try Rules -> Backtrack if Fail) / (If Earley: Fill Chart States [S1, S2...Sn] -> Avoid Redundancy).

Metric	Top-Down	Earley
Handling Left Recursion	Infinite Loop / Fails	Fully Supported
Efficiency	Exponential (Worst Case)	Polynomial $O(n^3)$
Suited for	Simple, small grammars	Large, ambiguous, real-time input

4. Justification

Earley parsing is the superior choice for dynamic real-time conditions. Its ability to handle all types of CFGs and its lack of backtracking mean that even if a user speaks an ambiguous or partial sentence, the system remains stable and efficient, providing the best possible partial analysis without a performance hit.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.

Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

Natural language is inherently ambiguous. The sentence 'She saw the man with a telescope' is a classic example of Prepositional Phrase (PP) attachment ambiguity.

1. CFG (Context-Free Grammar)

A standard CFG will generate two distinct trees: one where the telescope belongs to the man (VP -> V NP; NP -> NP PP) and one where she uses the telescope (VP -> VP PP). However, a pure CFG has no mechanism to decide which tree is more likely. It presents the developer with a 'parse forest' but no answer.

2. PCFG (Probabilistic CFG)

PCFG adds a probability to every rule (e.g., S -> NP VP [0.9]). By multiplying the probabilities of all rules used in a tree, the parser can rank them. If data shows that 'saw with' is more common than 'man with', the PCFG will pick the correct interpretation. However, it still lacks deep semantic context.

3. Neural Parsing

Neural parsers use vector embeddings (BERT, etc.) to capture the 'meaning' of words. It doesn't just look at rule frequencies; it understands that 'telescope' is an instrument for 'seeing'. This deep context allows it to resolve ambiguity with much higher accuracy than statistical rule-based systems.

4. Flow Chart & Table

Flow Chart: [Ambiguous Sentence] -> [Parser] -> (CFG: Multiple Trees) -> (PCFG: Highest Prob Tree) -> (Neural: Semantic Context Logic).

Feature	PCFG	Neural Parsing
Logic	Frequency-based	Context/Embedding-based
Ambiguity	Calculates Probabilities	Learns Semantic Patterns
Performance	Good	State-of-the-art

5. Justification

Neural parsing is the most effective for real-world applications. Ambiguity resolution often requires 'world knowledge' (knowing what a telescope is). While PCFGs are a step up from basic CFGs, only neural models can truly weigh the semantic context to determine the intended meaning in complex sentences.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.

Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

A grammar system must ensure 'agreement' (e.g., singular subjects with singular verbs) and 'subcategorization' (verbs getting the right arguments).

1. Subcategorization Frames

Subcategorization frames are lists associated with verbs that specify what kind of arguments they take. For example, 'find' needs an NP object, but 'want' can take an NP or an infinitive phrase. While useful, these frames are isolated to verbs and don't easily handle features like gender or number agreement across a whole sentence.

2. Feature Structures & Unification

Feature structures are Attribute-Value Matrices (AVMs) that represent linguistic properties like [NUMBER: SG, PERSON: 3rd]. Unification is the process of merging two structures. If one node says the subject is 'SG' and another node says the verb is 'PL', unification fails, and the sentence is rejected as ungrammatical. This is a powerful, unified way to enforce all constraints (case, gender, number, etc.) across the entire parse tree.

3. Flow Chart of Unification

Flow Chart: [Subject: NUM SG] + [Verb: NUM SG] -> [Unification] -> (Match -> Accept) / (Mismatch -> Reject).

Approach	Logic	Support for Agreement
Subcategorization	Verb-specific argument lists	Limited / Local
Feature Structures	AVM & Unification	Comprehensive / Global

4. Justification

Feature structures provide better support for grammatical correctness. They allow for a single mechanism (unification) to handle everything from verb arguments to gender and number agreement. This reduces the complexity of the grammar rules while increasing the precision of the system's correctness checks.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

In large-scale NLP, such as indexing the web or processing big data, the parser must be extremely fast without sacrificing too much accuracy.

1. Transition-Based Parsing

Transition-based parsing uses a stack-based machine. It makes a sequence of local decisions (Shift, Left-Arc, Right-Arc) to build a dependency tree. It is extremely fast, with a linear time complexity $O(n)$. However, because it makes greedy decisions at each step, it can sometimes miss the 'big picture' of a sentence, leading to error propagation.

2. Graph-Based Parsing

Graph-based parsing views the sentence as a complete graph where every word could potentially depend on any other word. It uses algorithms like the Maximum Spanning Tree (MST) to find the most likely global structure. It is more accurate than transition-based parsing but much slower ($O(n^2)$ or $O(n^3)$).

3. Comparative Flow Chart

Flow Chart: [Big Dataset] -> (Choice A: Transition-Based -> Fast Linear Scan -> Sequential Stack Decisions) / (Choice B: Graph-Based -> Global Matrix Optimization -> Slow/High Accuracy).

Criterion	Transition-Based	Graph-Based
Time Complexity	Linear $O(n)$	Polynomial $O(n^2)$
Decision Strategy	Local / Greedy	Global / Optimal
Use Case	Large-scale / Big Data	High-precision Research

4. Justification

For large-scale applications, transition-based parsing is the justified choice. When dealing with massive linguistic datasets, the linear performance allows the system to scale effectively. While graph-based methods are more accurate, the time penalty is too high for processing billions of sentences in a production environment.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature:	Signature:
Date: _____	Date: _____	Date: _____